

This looks like a complete **DevOps Lab Practical List**. I'll explain each experiment in very simple language, covering:

1. **What it is (Terminology)**
2. **Why it is used**
3. **How it works**
4. **Step-by-step implementation**

1. Implementation of DevOps Related Linux Commands

What is Linux?

Linux is an Operating System used on servers.

Most cloud servers, Docker containers, Jenkins servers, and DevOps tools run on Linux.

Basic Linux Commands

Check Current Folder

```
pwd
```

Output:

```
/home/abhinandan
```

Meaning: Shows where you currently are.

List Files

```
ls
```

Shows all files and folders.

Create Folder

```
mkdir DevOpsLab
```

Creates a folder.

Enter Folder

```
cd DevOpsLab
```

Moves inside folder.

Create File

```
touch file.txt
```

Creates an empty file.

View File

```
cat file.txt
```

Displays file content.

Copy File

```
cp file.txt backup.txt
```

Copies file.

Rename File

```
mv file.txt newfile.txt
```

Renames file.

Delete File

```
rm newfile.txt
```

Deletes file.

Clear Terminal

```
clear
```

Clears screen.

2. Implementation of Staging, Commit and Push on GitHub

Terminology

Git

Version Control System.

Tracks code changes.

GitHub

Online platform where Git repositories are stored.

Repository (Repo)

Project storage.

Staging

Preparing files before commit.

Commit

Saving changes locally.

Push

Uploading changes to GitHub.

Step-by-Step

Open VS Code

Open project folder.

Open Terminal

Ctrl + `

Initialize Git

```
git init
```

Output:

```
Initialized empty Git repository
```

Check Status

```
git status
```

Add File

```
git add .
```

or

```
git add index.html
```

Moves files to staging area.

Commit

```
git commit -m "First Commit"
```

Creates local snapshot.

Connect GitHub Repository

```
git remote add origin https://github.com/username/project.git
```

Push

```
git push -u origin master
```

Uploads code to GitHub.

3. Commit, Modification on Branch, Forking and Repository Link

Branch

A separate version of project.

Allows testing without affecting main code.

Create Branch

```
git branch feature1
```

Switch Branch

```
git checkout feature1
```

Modify File

Example:

```
<h1>Hello DevOps</h1>
```

Save file.

Commit Changes

```
git add .  
git commit -m "Modified homepage"
```

Push Branch

```
git push origin feature1
```

Forking

What is Fork?

Copy someone else's repository into your GitHub account.

Used in Open Source Projects.

Steps

1. Open GitHub Repository

2. Click Fork
3. Repository copied to your account

Example:

Original Repo

`github.com/company/project`

Forked Repo

`github.com/abhinandan/project`

4. Continuous Integration / Continuous Development with Jenkins and Java

Terminology

CI (Continuous Integration)

Automatically build and test code after every commit.

CD (Continuous Delivery)

Automatically deploy code.

Jenkins

Automation server.

Used to:

- Build code
 - Test code
 - Deploy code
-

Installation

Install Java

Check:

```
java --version
```

Install Jenkins

Ubuntu:

```
sudo apt update  
sudo apt install jenkins
```

Start Jenkins:

```
sudo systemctl start jenkins
```

Check Status:

```
sudo systemctl status jenkins
```

Access Jenkins

Open Browser:

```
http://localhost:8080
```

Create Jenkins Job

New Item

Click:

New Item

Select:

Freestyle Project

Connect GitHub

Paste Repository URL.

Build Step

```
echo "Building Project"
```

Save

Build Now

Jenkins automatically executes build.

5. GitHub Cloning

What is Clone?

Downloads repository from GitHub.

Copy Repository URL

Example:

```
https://github.com/user/project.git
```

Open Terminal

```
git clone https://github.com/user/project.git
```

Output:

```
Cloning into 'project'
```

Move inside:

```
cd project
```

6. Docker Commands

What is Docker?

Docker packages application + dependencies together.

Works same on every machine.

Check Docker

```
docker --version
```

Download Image

```
docker pull nginx
```

View Images

```
docker images
```

Run Container

```
docker run nginx
```

Run in Background

```
docker run -d nginx
```

View Running Containers

```
docker ps
```

Stop Container

```
docker stop container_id
```

Remove Container

```
docker rm container_id
```

Remove Image

```
docker rmi nginx
```

7. Containerization Using Docker

What is Containerization?

Packaging application with:

- Code
- Libraries
- Dependencies

into one container.

Example

Create Java File

```
public class Hello {  
    public static void main(String[] args){  
        System.out.println("Hello Docker");  
    }  
}
```

Dockerfile

```
FROM openjdk:17
```

```
COPY . /app  
WORKDIR /app  
RUN javac Hello.java  
CMD ["java", "Hello"]
```

Build Image

```
docker build -t hello-app .
```

Run Container

```
docker run hello-app
```

Output:

```
Hello Docker
```

8. Maven Build Life Cycle

What is Maven?

Java Build Tool.

Used for:

- Dependency Management
 - Build Automation
 - Packaging
-

Important File

pom.xml

Contains:

- Project Name
- Dependencies
- Plugins

Maven Life Cycle

Validate

Checks project.

```
mvn validate
```

Compile

Converts Java code into bytecode.

```
mvn compile
```

Test

Runs test cases.

```
mvn test
```

Package

Creates JAR file.

```
mvn package
```

Output:

```
target/project.jar
```

Install

Stores JAR in local repository.

```
mvn install
```

Clean

Deletes build files.

```
mvn clean
```

Flow

```
Source Code
  ↓
Validate
  ↓
Compile
  ↓
Test
  ↓
Package (.jar)
  ↓
Install
```

9. Selenium Installation and Automated Testing

What is Selenium?

Automation Testing Tool.

Used to automatically:

- Open Browser
 - Click Buttons
 - Fill Forms
 - Verify Results
-

Installation

Create Maven Project

VS Code → Open Folder

Add Dependency in pom.xml

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.21.0</version>
</dependency>
```

Download Chrome Driver

Version should match Chrome Browser.

Example Program

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class Test {
    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();

        driver.get("https://google.com");

        System.out.println(driver.getTitle());

        driver.quit();
    }
}
```

Execution Flow

```
Open VS Code
  ↓
Open Terminal
  ↓
Create Project
  ↓
Initialize Git
  ↓
Push to GitHub
  ↓
Clone Repository
  ↓
Create Branch
  ↓
Commit Changes
  ↓
Install Java
  ↓
```

Install Maven
↓
Build Project
↓
Install Jenkins
↓
Connect GitHub
↓
Automatic Build
↓
Create Dockerfile
↓
Build Docker Image
↓
Run Container
↓
Install Selenium
↓
Execute Automated Test

This sequence is exactly how a beginner-friendly DevOps workflow is typically demonstrated in a lab or internship project.